

CSE320:SOFTWARE ENGINEERING

L:3 T:0 P:0 Credits:3

Course Outcomes: Through this course students should be able to

CO1 :: Explain the evolution of Software Engineering and apply SDLC models to analyze requirement-based problems.

CO2 :: Apply software design principles to evaluate and analyze suitable system architectures for given scenarios.

CO3 :: Use UML models to represent systems and analyze their correctness and consistency.

CO4 :: Apply software testing techniques and analyze test results to determine software quality.

CO5 :: Use project management and DevOps tools to analyze project planning, estimation, and process efficiency.

CO6 :: Explain software quality standards and emerging technologies and analyze their impact on software development and maintenance

Unit I

Software Engineering Foundations & SDLC : Evolution and impact of software engineering, Software development life cycle (SDLC), Life cycle models (Waterfall, Prototyping, Spiral, V-Model Agile & Scrum DevOps lifecycle), Functional & Non-functional requirements, Requirement gathering & analysis, Software Requirements Specification (IEEE standard, writing & validation)

Unit II

Software Design Principles & System Architecture : Basic principles of software design, Modularity, Cohesion, Coupling, Types of cohesion and coupling, Measuring module independence, Design trade-offs, Function-oriented design: Data Flow Diagrams (DFD) - symbols, notations and leveling, Context diagrams and decomposition and Rules for constructing DFDs, Structure Charts - components and notation, Module hierarchy and control flow and Transform and transaction analysis, Design documentation and design review techniques

Unit III

Object-Oriented Software Development and Modeling Techniques : Unified Process: Phases and core workflows, Iterative and incremental development, UML Modelling: Use Case Diagrams, Class Diagrams, Sequence Diagrams, Activity Diagrams, Object modelling process, Model validation: Consistency and completeness checking, Traceability from requirements, Coding standards and code review techniques

Unit IV

Software Testing Concepts, Techniques, and Automation : Fundamentals of software testing, Functional and Non-Functional Software Testing, Testing Techniques: Black box, White box, Boundary value, Equivalence partitioning, Levels of testing: Unit, Integration, System, UAT, Types of Software Testing: API Testing, Web Testing, Mobile Testing, Automation Testing: Selenium IDE: Installation, record & playback tests, Introduction to Selenium WebDriver (conceptual), Performance Testing basics, Security Testing basics, Introduction to AI-assisted testing tools (overview)

Unit V

Software Project Management & DevOps Practices : Project management basics, Project planning & monitoring, Cost estimation methods (Function Points, Use Case Points, COCOMO (intro), Software Configuration Management (SCM), Introduction to CI/CD tools (GitHub Actions, GitHub CI/CD workflows), Scheduling Techniques (CPM, PERT, Gantt Charts)

Unit VI

Quality Management, Maintenance & Emerging Technologies : Quality management & standards: ISO 9001, SEI CMMI, Six Sigma & PSP, Computer-Aided Software Engineering (CASE tools), Software maintenance: types & challenges, Software reuse & Component-Based Software Development (CBSD), Advanced and Future Techniques: Cloud-native software development, AI in software development (GitHub Copilot, Code Whisperer), Low-code / No-code platforms

Text Books:

1. FUNDAMENTALS OF SOFTWARE ENGINEERING by RAJIB MALL, PRENTICE HALL
2. SOFTWARE ENGINEERING:A PRACTITIONER APPROACH by ROGER S.PRESSMAN, MCGRAW HILL EDUCATION
3. FUNDAMENTALS OF SOFTWARE ENGINEERING by RAJIB MALL, PRENTICE HALL

References:

1. SOFTWARE ENGINEERING FUNDAMENTALS by ALI BEHFAROOZ AND FREDERICKS J. HUDSON, OXFORD UNIVERSITY PRESS
2. SOFTWARE ENGINEERING by IAN SOMMERVILLE, PEARSON